01 · OVERVIEW

8:40 – 9:00 · 20 minutes

Overview of AI/HPC Distributed Storage

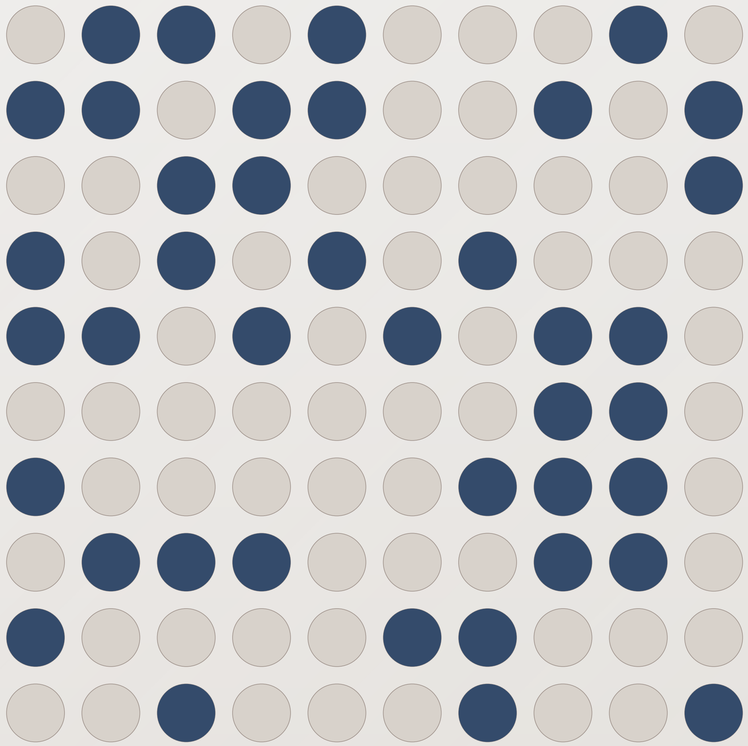
Architectures · data paths · file systems · object storage

Presenter: Christopher Grossmeier, Director of Product, DDN

ARCHITECTURES | DATA PATHS | FILE SYSTEMS | OBJECT

You bought 100 GPUs. Are they working?

100 GPUS · ONE CLUSTER



40%

average utilization

working

waiting on data

AI clusters are no longer compute-bound. They're I/O-bound.

1. **Data doesn't arrive fast enough**
   the pipe to the GPU is the limit, not the GPU

2. **Storage is misconfigured**
   the capability is there — the settings aren't

3. **The data path is inefficient**
   every extra hop is latency the GPU waits on

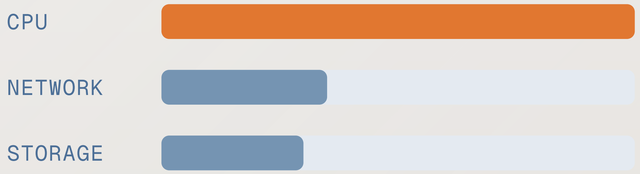
"store data reliably" → **"feed GPUs continuously."**

The moving bottleneck

2000S

CPU-bound

Single-node compute · HDD storage

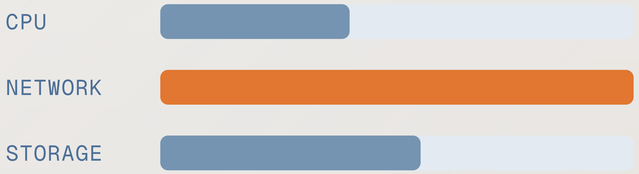


The processor was the wall.

2010S

Network-bound

Distributed systems · faster CPUs · SSDs

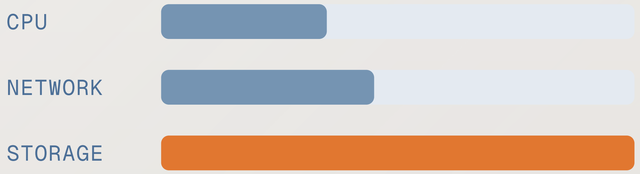


The wall moves to network + scaling.

TODAY

Storage-bound

GPU clusters (100s–1000s) · massive parallel I/O

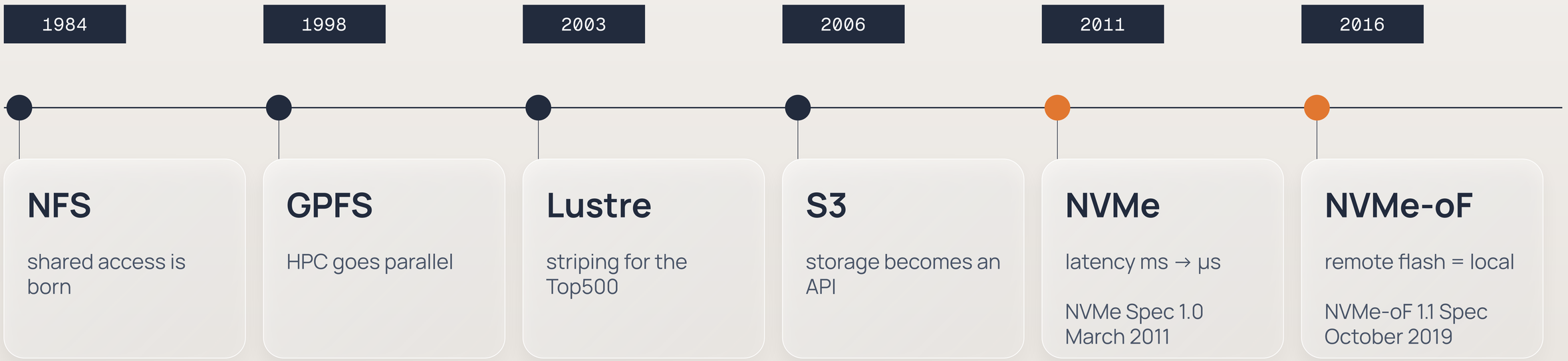


The wall is storage architecture + data path.

We solved CPU. We solved network. Now we're solving storage.

PCIe generations kept doubling for NICs and NAND — the fabric kept up. Shared-access design didn't.

1984–2018: thirty-five years of shared storage



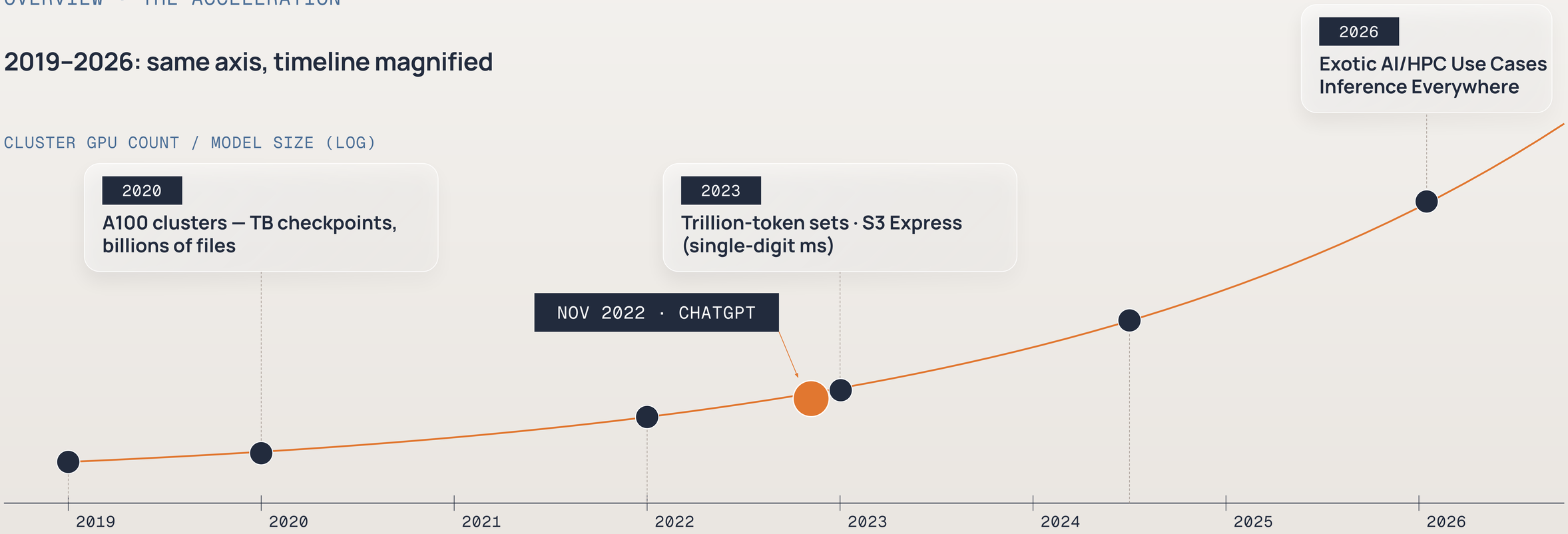
THE SETUP LINE

“Devices got a thousand times faster. Shared access at scale was never redesigned. That bill came due five years ago.”

HPC High-Performance Computing **NFS** Network File System **GPFS** General Parallel File System **S3** Simple Storage Service **NVMe-oF** NVMe over Fabrics **NVMe** Non-Volatile Memory Express

2019–2026: same axis, timeline magnified

CLUSTER GPU COUNT / MODEL SIZE (LOG)



← 1984-2018 compressed into this start point

2019
GPUDirect Storage — CPU exits the data path

2022
AI becomes THE workload

2024-25
Inference › training · KV-cache · object over RDMA

> “The supercomputer used to be the exotic use case for storage. After ChatGPT, the whole industry has our problems.”

AI Artificial Intelligence **S3** Simple Storage Service **RDMA** Remote Direct Memory Access **KV** Key-Value

What “parallel” actually means

PARALLEL FS

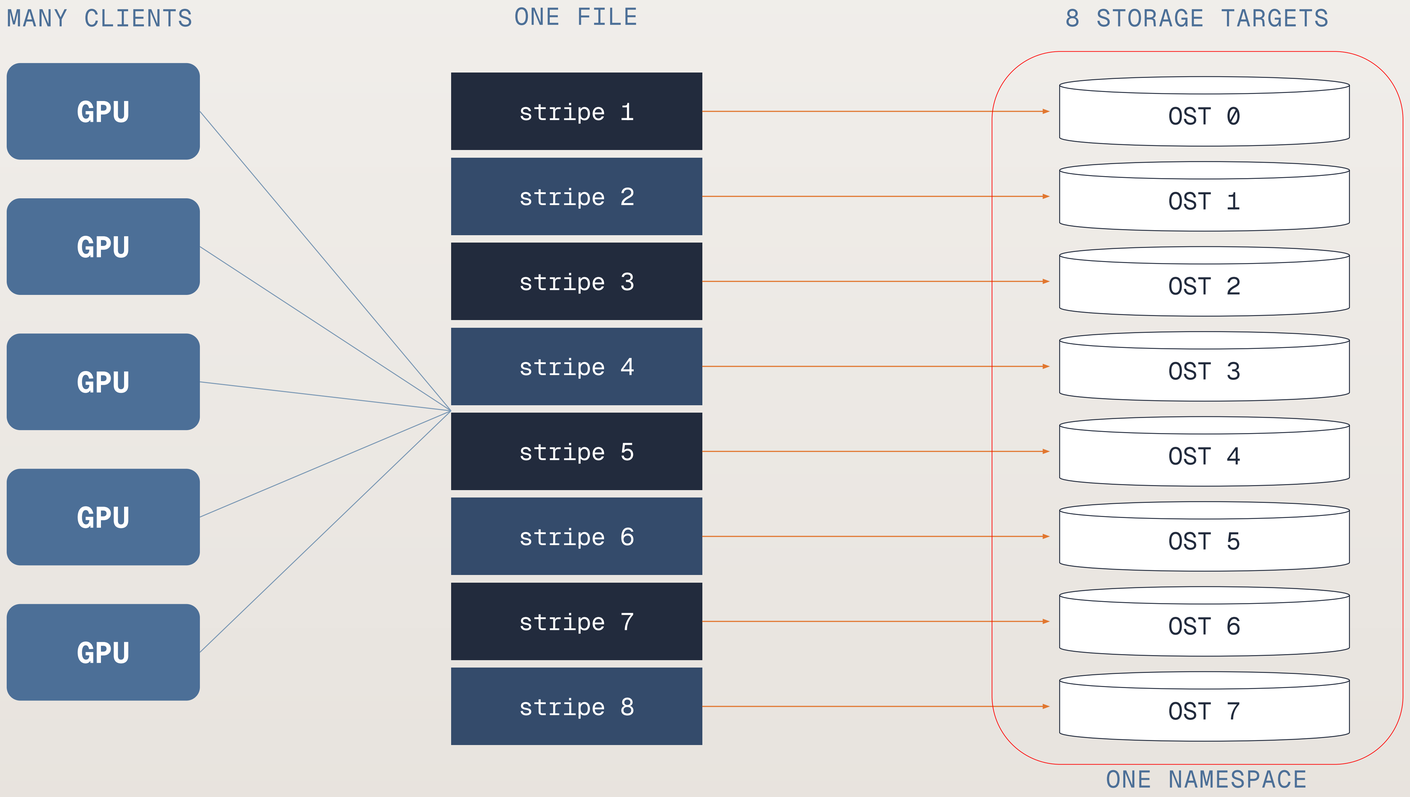
One file is striped across many storage targets; many clients read/write it simultaneously.

DISTRIBUTED SYSTEM

Storage, metadata, and clients are separate components — scale out by adding nodes.

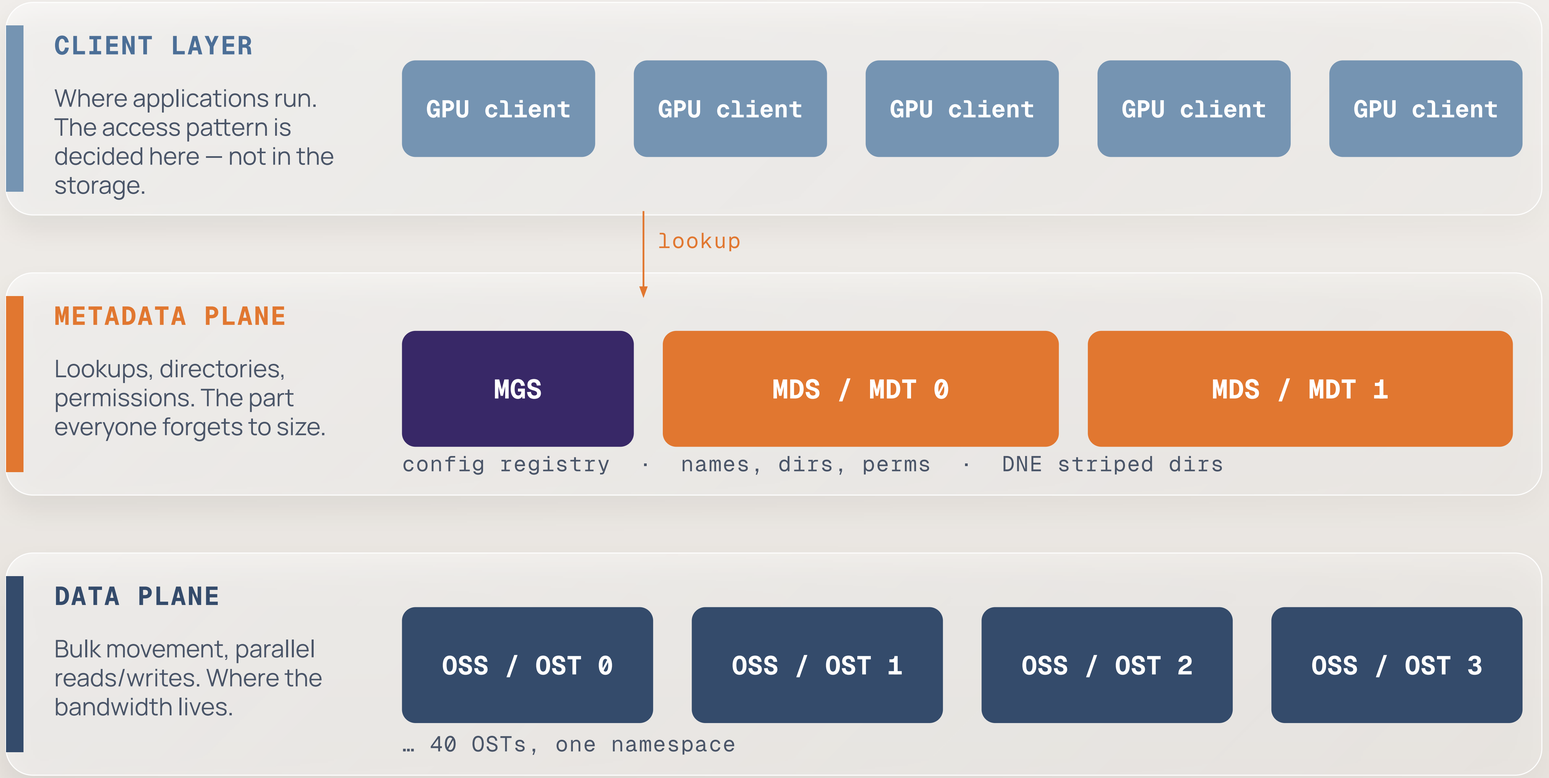
PARALLEL + DISTRIBUTED

Many clients · many nodes · many targets · ONE namespace.



“This is how you turn thousands of disks into one system — and thousands of GPUs into one workload.”

Three planes — remember these colors



FOUR MESSAGES OF THIS SECTION

1. **Storage devices alone are a bottleneck.**
2. **The problem is shared access at scale.**
3. **Parallel FS = striping + concurrency.**
4. **Metadata is critical — and overlooked.**

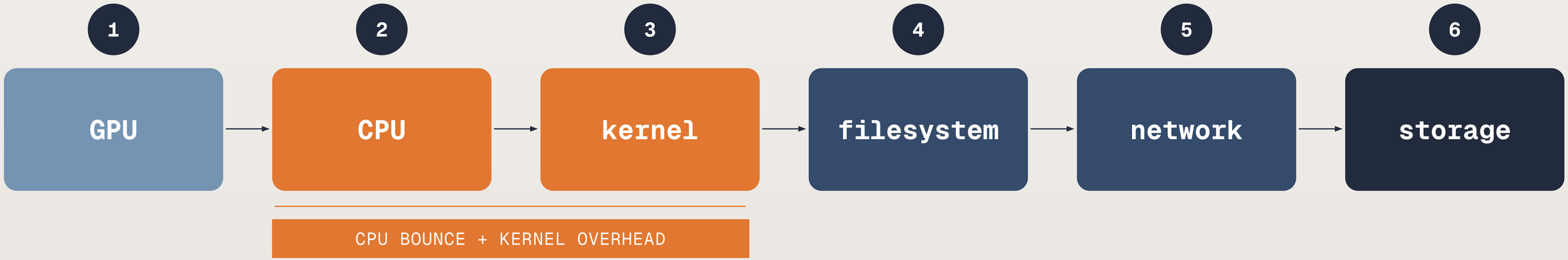
OST Object Storage Target **OSS** Object Storage Server **MDT** Metadata Target **MDS** Metadata Server **MGS** Management Server **DNE** Distributed Namespace **FS** File System

Each system answers a different era's question

SYSTEM	WHO / WHAT	WHEN TO USE IT	WHY IT WINS
NFS 1984	Simplicity, ubiquity	Home dirs, tools, shared config	**Deployed Everywhere: Simple zero friction**
GPFS / Spectrum Scale 1998	Balanced throughput + enterprise features	Mixed enterprise HPC	**Features + performance**
Lustre / EXAScaler 2003	Maximum parallel throughput, striping	Training data, multi-TB checkpoints	**Raw sequential bandwidth Performance at scale**
S3 / object storage 2006	API-driven, scale-out, no mount point	Datasets, checkpoints, archive → primary	**Every client talks to every node**
Disaggregated (DDN Infinia et al.) TODAY	Object-native, metadata-rich, multi-protocol	GPU-era access patterns	**Built for how AI reads data**

EXAScaler is a hardened Lustre distribution; the mechanics here apply to any Lustre deployment.

Data path #1 — POSIX, what everyone assumes



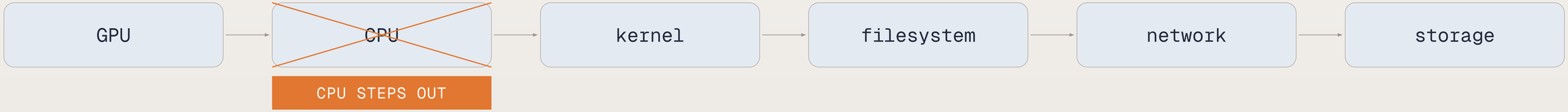
- **Bottleneck = CPU bounce + kernel overhead.**
- mmap acceleration helps (PyTorch / TensorFlow loaders lean on it) — but the CPU is still in the loop.
- Most frameworks default to this path — which is why defaults underperform.

6 hops

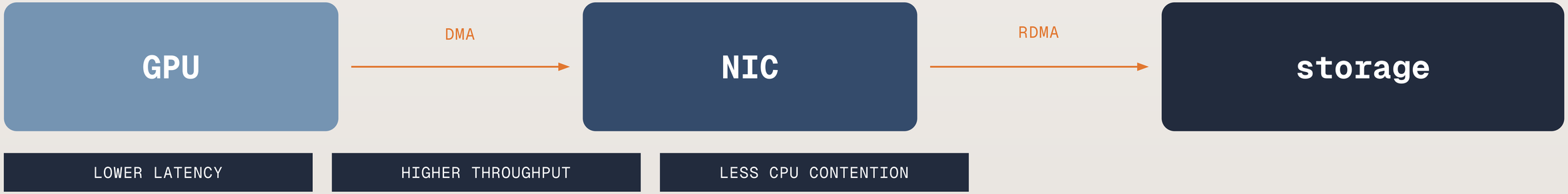
Every hop is latency the GPU waits on.

Data path #2 — GPUDirect / GDS, zero copy

POSIX · 6 HOPS



GPUDIRECT STORAGE · 2 HOPS



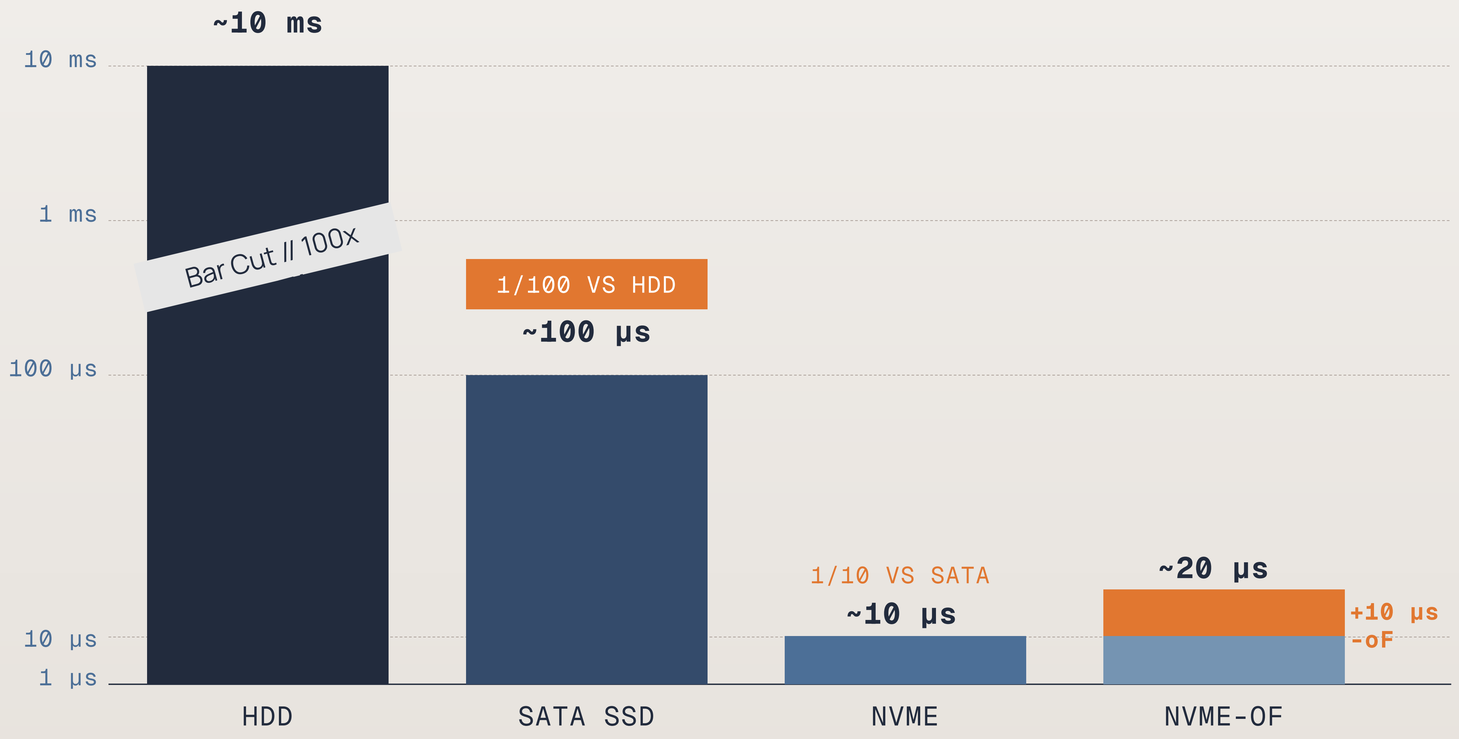
HANDS-ON HOOK

“Look at your PyTorch data loader — open/read/write, or a library that could use GDS?”

BRIDGE TO PROFILING

The profiler shows which path you’re actually on — that’s the first thing Lab 1 checks.

NVMe and NVMe-oF: remote flash ≈ local flash



latency, log scale — shorter bar = lower latency = better · order-of-magnitude figures

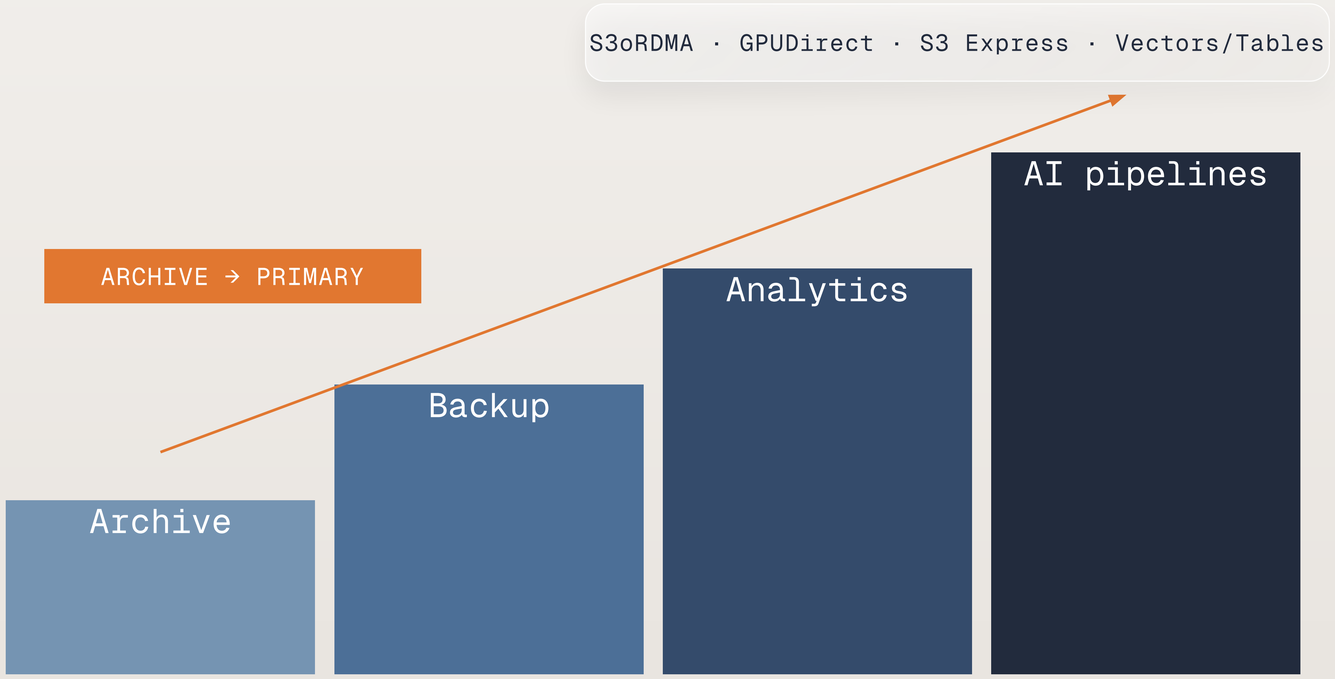
THE KEY IDEA

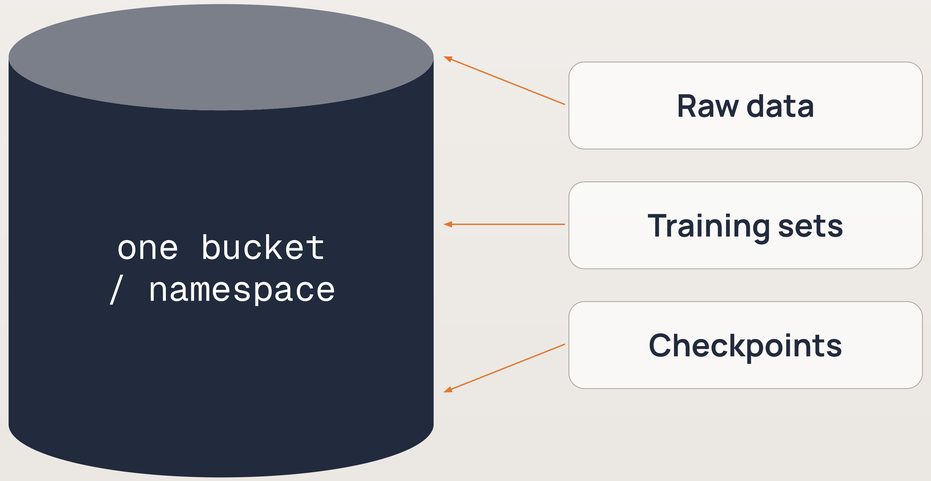
“Remote flash behaves (almost) like local flash.”

NVMe = device interface protocol.
NVMe-oF = its network extension.

Remote adds roughly one NVMe-worth of latency — about 10 µs over the fabric. That is what made disaggregated storage viable.

S3 grows up — archive to primary





API-driven, not mount-driven: every client talks to every node — no mount point to saturate.

- **Automated lifecycle tiering** cools cold data automatically.
- **Object is the default landing zone** for new cloud-native data — same API on-prem and in the cloud.
- **On-prem object is real now** same API — Infinia, Ceph, MinIO and others.

The storage system is no longer behind the GPU. It is part of the compute pipeline.

NEXT: HANDS-ON LAB 1 — PROFILING A CHECKPOINT-SAVE WORKLOAD IN NSIGHT SYSTEMS

1

Devices alone bottleneck

2

Shared access at scale is the problem

3

Parallel FS = striping + concurrency

4

Metadata is critical & overlooked